

# Il filesystem

Lezione 3 - comandi

## Sistemi Operativi - Il Modulo (Canale A - L)

**Prof. Paolo Zuliani**

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



**SAPIENZA**  
UNIVERSITÀ DI ROMA



# Agenda

- Continuiamo con i comandi per la gestione del file system, ovvero di file e directory



# Riepilogo Lezione 02

- inode e attributi file;
- come visualizzare attributi file: ls, stat, tree
- come creare file e directory
- File system, mounting
- passwd, group
- permessi accesso ai file
- chmod, chown, chgrp



# L03 - Comandi

*Consultate il man per i dettagli soffermandovi sulle opzioni indicate*

- `umask [mode]` la descrizione è nel man bash (cercate umask)
- `cp [-r] [-i] [-a] [-u] {filesorgenti} filedestinazione`
- `mv [-i] [-u] [-f] {filesorgenti} filedestinazione`
- `rm [-f] [-i] [-r] {file}`
- `ln [-s] sorgente [destinazione]`
- `touch [-a] [-m] [-t timestamp] {file}`
- `du [-c] [-s] [-a] [-h] [--exclude=PATTERN] [files...]`
- `df [-h] [-l] [-i] [file]`
- `dd [opzioni]`
- `mkfs [-t type fsoptions] device`

# umask [mode]

- Imposta la maschera dei file a `mode`; ovvero i diritti di accesso al file o alle directory nel momento della loro creazione
- Diritti accesso directory =  $0777 \text{ AND NOT}(\text{umask})$ 
  - L'operazione AND è bit a bit
  - 0666 per i file
  - Gli attributi speciali non sono impostati (notare il primo 0 ottale = 000)
- Per i file però il diritto di esecuzione non viene impostato (e quindi le opzioni speciali non hanno effetto) [notare  $666=110110110$ ]
- **Esercizio 1:** modificare l'umask in modo che, sia che venga creata una directory sia un file, i permessi siano 664. Dopodiché, modificare nuovamente in modo che il permesso per una nuova directory sia 775 e per un file sia 664.

```
cp [-r] [-i] [-a] [-u]  
{filesorgenti} filedestinazione
```

- -r recursive per directory
- -i interactive per essere avvisati in caso di sovrascrizione
- -u la sovrascrittura avviene solo se l'mtime del sorgente è più recente di quello della destinazione
- **Esercizio 2:** verificare il funzionamento di ciascuna delle opzioni mostrate. Per esempio, per l'opzione -i, provare a copiare un file sorgente in un file destinazione esistente e vedere che effettivamente viene chiesta conferma. Per l'opzione -a, è sufficiente creare un file ed aggiungere qualche permesso, poi fare la copia con e senza -a...
- **Esercizio 3:** verificare il funzionamento del comando cp con diverse tipologie di argomenti: i) 2 file (tutti e 4 i casi tra esistenti e non); ii) 2 directory (tutti e 4 i casi tra esistenti e non); iii) 3 file (l'ultimo sia esistente che non); iv) 3 directory (idem al caso precedente); v) 2 file e 2 directory...

```
mv [-i] [-u] [-f] {filesorgenti}  
filedestinazione
```

- Sposta un file (o lo rinomina)
- -i e -u hanno stesso significato di `cp`
- -f force è il default
- **Esercizio 4:** rifare gli esercizi 2 e 3 con `mv` al posto di `cp`

```
rm [-f] [-i] [-r] {file}
```



- Ricordate che non c'è il cestino!!!!
- -f forza la cancellazione (senza chiedere)
- -r recursive
- -i interactive
- **Esercizio 5:** creare un file, guardare il suo inode, cancellarlo e poi creare subito dopo un altro file: quale è l'inode del nuovo file (supponendo che nessun altro utente stia usando il computer...)?



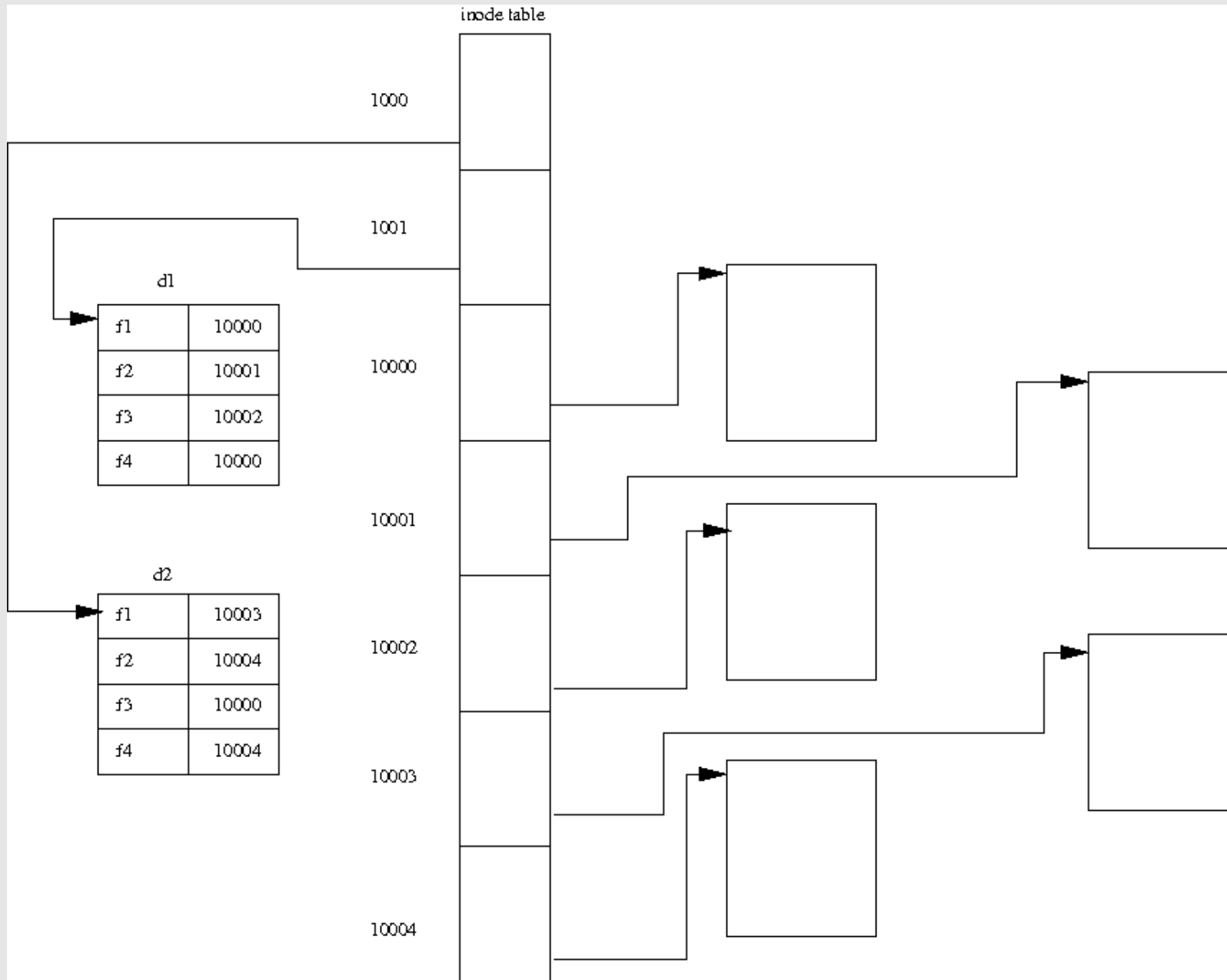
`ln [-s] sorgente [destinazione]`

- Soft/symbolic e hard link
- Un link `lin1` ad un file `f1` è un *nuovo* file `lin1` che punta ad `f1` senza contenere in esso il contenuto di `f1` (permette di risparmiare spazio)
- Default hard link; `-s` soft link
- Negli hard link non c'è il concetto di puntatore e puntato

# Esercizio 7

- Eseguite i seguenti comandi e verificate che si ottiene quanto mostrato nella figura mostrata nella slide seguente (usate `stat` o `ls`)

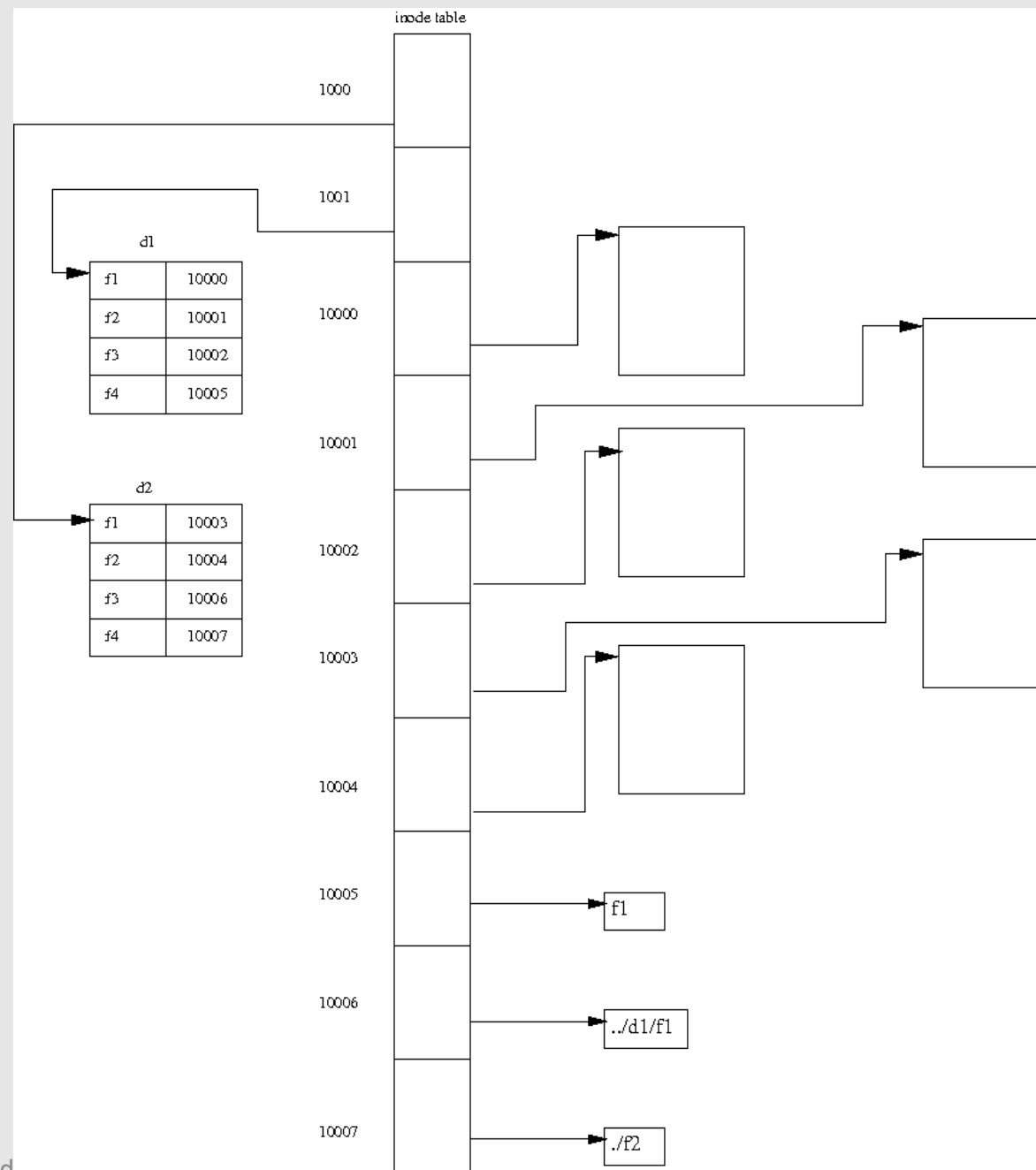
```
mkdir d1 d2
dd if=/dev/urandom of=d1/f1 bs=1M count=10
dd if=/dev/urandom of=d1/f2 bs=1M count=10
dd if=/dev/urandom of=d1/f3 bs=1M count=10
dd if=/dev/urandom of=d2/f1 bs=1M count=10
dd if=/dev/urandom of=d2/f2 bs=1M count=10
ln d1/f1 d1/f4
ln d1/f1 d2/f3 #sarebbe stato lo stesso ln d1/f4 d2/f3
ln d2/f2 d2/f4
```



zulliani@di.uniroma1.it

# Esercizio 8

- Nell'Esercizio 7 sostituite ogni `ln` con `ln -s` e verificate che si ottiene la figura mostrata a lato (usate `stat` o `ls`)
- Attenzione: `ln -s` NON controlla se il file sorgente esiste!! Per gli hard link il controllo viene invece effettuato. [Perché?]



- A seguito dell'esercizio svolto, che differenza avete notato tra link Hard e Soft?
- I link funzionano anche per le directory.

```
touch [-a] [-m] [-t timestamp]
{file}
```

- Serve per modificare i timestamps («marcature/timbri temporali») di un file ... e se il file non esiste lo crea
- Può essere applicato anche su directory
- -t imposta il timestamp desiderato
- **Esercizio 9:** immaginare almeno due casi in cui touch non riesce a creare un file

```
du [-c] [-s] [-a] [-h] [--exclude=PATTERN]  
[files...]
```

- Somma tutte le dimensioni dei file e/o directories dati come argomento
- Studiate le opzioni indicate sopra (leggendo il man)
- **Esercizio 10:** verificare le opzioni sopra riportate su alcune directory standard di Linux (ad es., /etc)

```
df [-h] [-l] [-i] [file]
```

- Mostra la dimensione e l'attuale uso dei filesystem
- Studiate le opzioni indicate sopra (leggendo il man)
- **Esercizio 11:** vedere le statistiche di uso di tutti i filesystem, e poi solo del filesystem contenente i seguenti files (o folder): `/etc/passwd` e `/proc`



# dd [opzioni]

- Per convertire o copiare files in modo «sophisticato»
- Le opzioni sono una sequenza `variabile=valore`
- Le variabili più importanti sono:
  - `bs` dimensione di un singolo blocco in lettura/scrittura
  - `count` numero di blocchi da copiare
  - `convert`, in questo caso, il valore specifica una conversione, per esempio di codifica (da minuscolo a maiuscolo e viceversa, più svariate altre cose; vedere il man)
  - `if` file di input (se non dato, legge da tastiera (standard input))
  - `of` file di output (se non dato, scrive su schermo (standard output))
- Oltre che per le conversioni si usa per copiare file speciali che non possono essere copiati con `cp`

# dd (cont.)

- Quando potrebbe essere utile usare `dd` nel modo appena visto?
  - Cancellare dati completamente da un supporto di memoria (e.g., sovrascrivere le vostre informazioni private con dati casuali)
  - Preparare un file ad essere formattato
  - Copiare una parte di un file



- **Esercizio 12:** creare un file di testo, con il seguente contenuto:

ciao1

addio2

via3

ehila4

dove vai5

e poi copiarlo in un altro file in modo tale che quest'ultimo contenga solo i bytes che vanno dal decimo al ventesimo; usare sia 1 che 10 come valori per bs; rifare nuovamente la copiatura sullo stesso file destinazione, ma questa volta fare in modo che il contenuto del file destinazione sia duplicato (ovvero, contenga per 2 volte consecutive i bytes che vanno dal decimo al ventesimo)

# `mkfs [-t type fsoptions] device`

- Crea un filesystem su device; a volte detto anche formattazione perché prepara i file a memorizzare file secondo un dato formato, ad es. `ext4`
- I tipi `type` li abbiamo visti la scorsa lezione – vedi anche il man
- `fsoption` può essere `ro` oppure `rw`
- `device`: device file in `/dev` oppure file regolare, ad es. creato con `dd`
  - Se è un dispositivo, non deve essere montato (`/dev/hda`, `/dev/hda2`, `/dev/sdb1` )
- Se nella vostra distro non è disponibile `mkfs` usare `/sbin/mkfs`



# Eseguire i seguenti comandi ad esempio nella home

```
dd if=/dev/zero bs=1M count=100 of=fs-virtuale
ls -l fs-virtuale
sudo mkfs -t ext3 fs-virtuale
mkdir dir
sudo mount -t ext3 fs-virtuale dir
# prima di continuare controllare ed eventualmente sistemare
# i diritti di accesso a dir
cd dir
ls -l
dd if=/dev/zero bs=1M count=2 of=file1
cd ..
ls -l
rm -rf dir
cd dir
dd if=/dev/zero bs=1M count=98 of=file2
cd ..
umount dir
ls -l dir
rm -rf dir
sudo mount -t ext3 fs-virtuale dir
cd dir
ls -l
umount .
cd ..
umount dir
```

Richiede sudo



# Domande?

